CLAUDE CODE BEGINNER GUIDE

Written for high school students, parents, and anyone who has never used a terminal before. Twelve parts covering everything from installation to publishing.

Mac + Cursor (free) | Zero experience required | 12 parts + reference

February 2026

01

INSTALLATION

Think of this like setting up a game. There are three things you need, and they all work together:

Claude Pro -- a subscription ($20/month, or start a free trial) from Anthropic. This is like buying a ticket that gives you access to Claude's brain. Without it, Claude Code won't work.

Claude Code -- the actual AI assistant that lives inside your terminal (the text window on your computer). You type questions and instructions in plain English, and it does the work.

Cursor -- a program that looks like a fancy notepad for writing code. It has a built-in terminal window at the bottom where you'll run Claude Code. The free version is all you need.

STEP 1 -- GET CLAUDE PRO

1. Go to **claude.ai**
2. Create a free account if you don't have one
3. Click your name or icon in the bottom left corner and choose **Upgrade to Pro**
4. Subscribe ($20/month) or start a free trial if you'd rather not pay right away

STEP 2 -- INSTALL CURSOR

1. Go to **cursor.com**
2. Click **Download** and install it like any other Mac app (drag it to your Applications folder)
3. Open Cursor -- it looks like a code editor with a file list on the left side

STEP 3 -- OPEN THE TERMINAL

The terminal is a plain text window where you talk to your computer by typing commands. It sounds scarier than it is -- you'll be using it constantly and it becomes second nature fast.

1. In Cursor, click **Terminal** in the top menu bar
2. Select **New Terminal**
3. A panel opens at the bottom of the screen with a blinking cursor

You'll see a line that looks like this:

```
yourname@MacBook ~ %
```

This is called your **command prompt.** It tells you you're ready to type. The `~` symbol means you're currently in your home folder.

STEP 4 -- INSTALL CLAUDE CODE

Type the following command exactly and press **Enter:**

```
curl -fsSL https://claude.ai/install.sh | bash
```

What this does: it goes out to the internet, downloads Claude Code, and installs it on your computer automatically.

After it finishes, close the terminal (click the trash icon or X on the terminal panel) and open a fresh one via Terminal > New Terminal. This is necessary so your computer recognizes the newly installed program.

Check that it worked by typing:

```
claude --version
```

You should see a version number printed back. If you see "command not found," open a completely new terminal window and try again.

STEP 5 -- LOG IN

Type:

```
claude
```

The first time, it will ask you to log in with your Anthropic account. It will open a browser window for you to confirm. Once you're in, you're inside Claude Code and can start talking to it.

To exit Claude Code at any time, type `/quit` and press Enter.

02 SETTING YOUR DEFAULT MODEL

Claude Code can use different versions of Claude's brain -- called **models.** The most powerful one is called **Opus.** The default is usually Sonnet, which is faster but slightly less capable.

THE ONE-TIME SETUP (RECOMMENDED)

This adds a single line to a hidden settings file that tells your computer "always use Opus when I launch Claude Code." Type this in the terminal:

```
echo 'export ANTHROPIC_MODEL="opus"' >> ~/.zshrc && source ~/.zshrc
```

To verify it worked, type:

```
echo $ANTHROPIC_MODEL
```

You should see `opus` printed back.

SWITCHING MODELS DURING A SESSION

If you're already inside Claude Code and want to switch models mid-conversation:

```
/model
```

This opens an interactive menu where you can choose between Opus, Sonnet, or Haiku using your arrow keys. Press Enter to confirm.

THE THREE MODELS

🧠 **OPUS**

The smartest, most thoughtful model. Best for complex planning, writing, and decision-making. Uses the most of your usage budget.

⚡ **SONNET**

A great middle ground. Fast and smart. Good for most everyday tasks. The default if you don't specify anything.

🐇 **HAIKU**

The quickest and cheapest. Good for simple, quick questions. Not great for anything complex.

Tip: There's also a special mode called **opusplan** -- it automatically uses Opus when planning and thinking, then switches to Sonnet when doing the actual writing or building. This saves your usage budget while still getting Opus's smart planning.

03 THE THREE WORKING MODES

This is one of the most important things to understand. Claude Code has three different modes that control how much it's allowed to do without asking you first. You switch between them by pressing **Shift + Tab** inside a running Claude Code session.

✋ **NORMAL MODE** DEFAULT

Claude asks your permission before doing almost anything -- creating files, running commands, making changes. **When you're learning or working on something important, stay here.**

▶▶ **AUTO-ACCEPT EDITS** SHIFT+TAB ×1

Claude automatically makes edits to files without stopping to ask each time. It still asks for permission before running bigger system commands. Think of it like letting a contractor work without watching every nail -- but you're still home if something big needs approval.

📋 **PLAN MODE** SHIFT+TAB ×2

Claude can only look at files and think -- it cannot make any changes at all. It reads everything and gives you a detailed plan. Think of it like asking an architect to describe what they'd renovate before any work begins.

THE FOURTH OPTION: DANGEROUSLY SKIP PERMISSIONS

`claude --dangerously-skip-permissions`

Claude gets permission to do absolutely everything without ever pausing to ask. Anthropic put "dangerously" in the name on purpose. Claude could accidentally delete files you didn't want deleted, edit something it shouldn't have, or go further than you intended.

Only use this for very well-defined, low-stakes tasks where you've clearly told Claude exactly what to do and what not to touch. A good rule of thumb: if you're not sure whether to use it, don't.

QUICK MODE REFERENCE

MODE	SHIFT+TAB PRESSES	WHAT CLAUDE CAN DO
Normal	0 (default)	Everything, but asks you first
Auto-Accept Edits	1 press	Makes file edits automatically; asks for bigger stuff
Plan Mode	2 presses	Can only read and think -- no changes at all
Skip All	Launch flag	Does everything with no questions asked (use carefully)

04 UNDERSTANDING YOUR FOLDERS

HOW FOLDERS WORK IN THE TERMINAL

Folders on your Mac (what you see in Finder) are the same folders the terminal sees. The only difference is how you write their

location.

In Finder you might click: **YourName > claudeprojects > my-project**

In the terminal, that same path is written as:

```
/Users/yourname/claudeprojects/my-project
```

The `~` symbol is a shortcut that means "my home folder." So you can also write the path above as `~/claudeprojects/my-project`.

THE CD COMMAND: HOW TO MOVE AROUND

`cd` stands for "change directory." A "directory" is just another word for a folder.

COMMAND	WHAT IT DOES
`cd claudeprojects`	Go into a folder
`cd ~/claudeprojects/my-project`	Go straight to a project
`cd ..`	Go back up one level (two dots = "go up one folder")
`cd ~`	Go all the way home from anywhere
`pwd`	Print working directory -- shows where you are
`ls`	List what's inside the current folder
`ls -a`	List everything, including hidden files (files starting with a dot)

THE HIDDEN .CLAUDE FOLDER

Your Mac hides files and folders that start with a dot (`.`). These are usually settings files that you're not supposed to accidentally mess with.

When Claude Code installs itself, it creates a folder called `.claude` inside your home folder. You'll never see it in Finder normally. It's not missing — it's just hidden.

Inside `.claude` is where Claude Code stores all its settings, your login credentials, and most importantly, a special instructions file called `CLAUDE.md` (explained in Part 10).

> **To peek at hidden files in Finder:** Navigate to your home folder and press **Cmd + Shift + .** (Command + Shift + Period). Hidden files and folders will appear grayed out. Press the shortcut again to hide them.
>
> **To see hidden files in the terminal:** Type `ls -a` and you'll see everything, including files starting with `.`

RECOMMENDED FOLDER STRUCTURE

```
yourname/                          <- your home folder, also written as ~
  .claude/                         <- hidden folder, created automatically
    CLAUDE.md                      <- your global rules for Claude (see Part 10)
    settings.json                  <- Claude Code's settings
  claudeprojects/                  <- your projects live here
    project-one/
      CLAUDE.md                    <- rules just for this project (optional)
      README.md                    <- what this project is
      SNAPSHOT.md                  <- save point you can return to
      tasks/
        todo.md                    <- Claude's step-by-step plan
        lessons.md                 <- mistakes learned, rules to prevent them
    project-two/
      README.md
      SNAPSHOT.md
      tasks/
        todo.md
        lessons.md
```

A companion visual for this is included as a separate file with this guide.

CREATING A NEW PROJECT

```
cd ~/claudeprojects
mkdir my-new-project
cd my-new-project
```

`mkdir` means "make directory" -- it creates a new folder.

STARTING CLAUDE CODE IN A PROJECT

Always navigate into your project folder before starting Claude Code. This is how Claude knows which project it's working on -- it reads the files in whatever folder you launched it from.

```
cd ~/claudeprojects/my-project-name
claude
```

05 TERMINAL SHORTCUTS

The terminal doesn't behave exactly like a normal text box. Here are the shortcuts that save the most frustration:

WHAT YOU WANT TO DO	SHORTCUT
Erase everything on current line	Ctrl + U
Erase from cursor to end of line	Ctrl + K
Jump to start of line	Ctrl + A
Jump to end of line	Ctrl + E
Move backward one word	Option + Left Arrow
Move forward one word	Option + Right Arrow
Clear the whole screen	Ctrl + L (or type clear)
Repeat your last command	Up Arrow

WHAT YOU WANT TO DO	SHORTCUT
Stop whatever is running	Ctrl + C

So yes — **Ctrl + U** clears the whole line. Very useful when you've made a typo halfway through a long command.

06 WHAT IS A .MD FILE?

A file ending in `.md` is a **Markdown file.** Markdown is a simple way to write text that can be formatted nicely — with headings, bold text, lists, and so on.

You write it in plain text using a few simple symbols:

YOU WRITE	IT LOOKS LIKE
`# Big Title`	A large heading
`## Smaller Heading`	A medium heading
`**bold text**`	bold text
`- list item`	a bullet point

The beauty of Markdown is that even without any special software, the file is completely readable as plain text. But tools like Cursor, GitHub, or Notion will display it beautifully formatted.

You'll use `.md` files constantly for notes, instructions, and documentation in your projects.

07 WHAT IS A README?

A `README.md` file is the welcome mat of any project. It lives in the main (root) folder of a project and is the first thing anyone reads to understand what the project is.

Claude Code reads your `README.md` automatically when you start a session. This means any instructions you write there will be followed every time — no need to re-explain things from scratch.

Think of the README as a standing briefing note for your AI assistant.

OTHER COMMON .MD FILES

FILE	WHAT IT'S FOR
CLAUDE.md	Instructions specifically for Claude's behavior
SNAPSHOT.md	A save point you can return to after closing the terminal
tasks/todo.md	A step-by-step plan with checkboxes
tasks/lessons.md	A record of mistakes and rules to avoid repeating them
CHANGELOG.md	A log of what changed and when

08 TOKENS & USAGE LIMITS

WHAT IS A TOKEN?

When you send a message to Claude, every word uses up a small amount of computing resource called a **token**. A token is roughly 3/4 of a word. So "hello" is about one token, and a paragraph of text might be 100 tokens.

Think of tokens like text messages: each one uses a small amount of your plan's allowance.

HOW CLAUDE PRO LIMITS WORK

Claude Pro gives you a generous monthly allowance. You'll almost never hit limits for normal use. But if you have extremely long conversations or send huge amounts of text repeatedly, you might see a message saying you've used up your current window.

To stay within limits:
• Start a fresh session for each new task instead of letting one conversation grow forever
• Use `/compact` to summarize a long session so Claude remembers the key points without storing every single word
• Opus uses more per message than Sonnet — use Sonnet for quick simple tasks

CONTEXT WINDOW

The "context window" is how much text Claude can hold in its working memory during a single conversation. Imagine it like short-term memory — if a conversation gets very long, the oldest parts start to fade out.

When this happens, Claude might seem to "forget" earlier things you said. The fix is to either use `/compact` to compress the history, or start a new session and point Claude to the README and SNAPSHOT files to catch it up.

09 THE SNAPSHOT SYSTEM

When you close the terminal, your Claude Code session ends. Any memory of what you discussed or built is gone. The snapshot system is how you make sure nothing is lost.

The idea is simple: you include instructions in your README telling Claude to save a summary file after every significant step. When you come back later, you show Claude that summary file and it picks up right where you left off.

HOW TO SET UP YOUR README

The easiest way is to drag this guide's PDF (or a screenshot of the template below) into the terminal in Cursor while you're inside your project folder, and tell Claude: "*create a README.md in this project using the example template, filled in with [a 2–3 sentence description of what you're building].*"

Claude will create the file with your project info and the snapshot instructions baked in.

README TEMPLATE (FOR REFERENCE)

This is the template Claude will use. You can read through it to see what gets added to your project:

```
# [Project Name]

## What This Project Is
[Write 2–3 sentences describing what this project does
or what you're building]

## Current Status
[Claude will update this each session]

## Instructions for Claude
Please do the following after every significant edit or decision:
1. Update the "Current Status" section above with what was just done
2. Update or create a file called SNAPSHOT.md in this folder containing:
   – The date of this snapshot
   – A summary of what has been built or decided so far
   – What the next steps are
   – Any important decisions made and why

When starting a new session, always read README.md and SNAPSHOT.md
first before doing anything.
```

When you come back after closing the terminal, just type:

```
"Please read README.md and SNAPSHOT.md before we begin."
```

Claude will read them and be fully caught up.

10 — CLAUDE.MD -- TEACHING CLAUDE HOW TO BEHAVE

WHAT IT IS

`CLAUDE.md` is the most powerful file in your whole setup. Claude Code reads it automatically at the start of every session -- before you say a single word. Whatever rules, preferences, or instructions you write in it become standing behavior.

There are two levels:

🌐 **GLOBAL**

`~/.claude/CLAUDE.md` -- applies to every project on your entire computer. This is where you put your universal rules -- like the workflow rules below.

📂 **PROJECT-LEVEL**

`~/claudeprojects/my-project/CLAUDE.md` -- applies only to that one project. Good for project-specific instructions or context.

HOW TO SET UP YOUR GLOBAL CLAUDE.MD

The easiest way is to drag and drop files or images directly into the terminal in Cursor and ask Claude to add their content to your global `CLAUDE.md`. Claude will create the hidden `.claude` folder and the `CLAUDE.md` file for you if they don't already exist.

Workflow rules: Drag the Workflow Orchestration screenshot (or this guide's PDF) into the terminal and tell Claude: "*add this content to my global CLAUDE.md.*"

Folder structure: Drag the Folder Structure Visual file into the terminal and tell Claude: "*add this folder structure to my global CLAUDE.md and follow it for all projects.*"

These additions aren't mandatory, but they're really nice to have for quality control and organization.

WORKFLOW RULES (FOR REFERENCE)

These are the workflow rules contained in the Workflow Orchestration screenshot. They're shown here so you can see exactly what gets added to your global `CLAUDE.md`:

```
# Claude's Working Rules

## Workflow Orchestration

### 1. Plan Mode Default
- Enter plan mode for ANY non-trivial task (3+ steps or architectural decisions)
- If something goes sideways, STOP and re-plan immediately -- don't keep pushing
- Use plan mode for verification steps, not just building
- Write detailed specs upfront to reduce ambiguity

### 2. Subagent Strategy
- Use subagents liberally to keep the main context window clean
- Offload research, exploration, and parallel analysis to subagents
- For complex problems, throw more compute at it via subagents
- One task per subagent for focused execution

### 3. Self-Improvement Loop
- After ANY correction from the user: update `tasks/lessons.md` with the pattern
- Write rules for yourself that prevent the same mistake
- Ruthlessly iterate on these lessons until mistake rate drops
- Review lessons at session start for the relevant project

### 4. Verification Before Done
- Never mark a task complete without proving it works
- Diff behavior between main and your changes when relevant
- Ask yourself: "Would a staff engineer approve this?"
- Run tests, check logs, demonstrate correctness

### 5. Demand Elegance (Balanced)
- For non-trivial changes: pause and ask "is there a more elegant way?"
- If a fix feels hacky: "Knowing everything I know now, implement the elegant
solution"
- Skip this for simple, obvious fixes -- don't over-engineer
- Challenge your own work before presenting it

### 6. Autonomous Bug Fixing
- When given a bug report: just fix it. Don't ask for hand-holding
- Point at logs, errors, failing tests -- then resolve them
- Zero context switching required from the user
- Go fix failing tests without being told how

## Task Management

1. **Plan First**: Write plan to `tasks/todo.md` with checkable items
2. **Verify Plan**: Check in before starting implementation
3. **Track Progress**: Mark items complete as you go
4. **Explain Changes**: High-level summary at each step
5. **Document Results**: Add review section to `tasks/todo.md`
6. **Capture Lessons**: Update `tasks/lessons.md` after corrections

## Core Principles
```

```
- **Simplicity First**: Make every change as simple as possible. Impact minimal code.
- **No Laziness**: Find root causes. No temporary fixes. Senior developer standards.
- **Minimal Impact**: Changes should only touch what's necessary. Avoid introducing bugs.
```

THE FULL FILE ECOSYSTEM

FILE	WHERE IT LIVES	WHAT IT'S FOR	WHO CREATES IT
~/.claude/CLAUDE.md	Hidden global folder	Universal rules for all projects	You (once)
CLAUDE.md	Inside a project	Rules for just that project	You (per project)
README.md	Inside a project	What the project is + snapshot prompt	You + Claude updates
SNAPSHOT.md	Inside a project	Session save point	Claude auto-updates
tasks/todo.md	tasks/ subfolder	Step-by-step plan with checkboxes	Claude creates + updates
tasks/lessons.md	tasks/ subfolder	Mistake log + rules to avoid repeating	Claude updates after corrections

11 WORKING FROM YOUR PHONE

Once you have a project running on your computer, you can mirror it into the Claude app on your phone — handy for picking up where you left off without being tied to your desk.

HOW TO TURN IT ON

Inside Claude Code (in your Cursor terminal), type:

```
/remote-control
```

That's it. Your project will automatically show up in the Claude app on your phone, and you can keep working on the same project from there.

12 PUBLISHING YOUR PROJECT

Before you publish: Your site will be public – anyone with the link can see it. Use your first name only. Don't include your phone number, home address, personal email, school name, or daily schedule.

WHAT IS NETLIFY?

Netlify is a free service that turns your project folder into a live website. You drag your folder onto a page, and it gives you a URL anyone can visit. No complicated setup, no public code repositories -- just drag, drop, done.

STEP 1 -- CREATE A FREE NETLIFY ACCOUNT

Go to **netlify.com** and click **Sign up.** You can sign up with an email address. The free plan is all you need. Creating an account means your site stays up permanently (without an account, sites expire after a short time).

STEP 2 -- GO TO NETLIFY DROP

Once logged in, go to **app.netlify.com/drop** in your browser. You'll see a large area that says to drag your folder here.

STEP 3 -- DEPLOY YOUR PROJECT

1. Open Finder and find your project folder (the one with `index.html` inside)

2. Drag the entire folder onto the Netlify Drop page
3. Wait a few seconds — Netlify will give you a live URL that looks like `random-name-12345.netlify.app`

That's your live website. Share that link with anyone and they can visit it in their browser.

UPDATING YOUR SITE LATER

1. Log into Netlify and go to your site's dashboard
2. Click the **Deploys** tab
3. Drag your updated project folder onto the deploy area
4. Your site updates instantly at the same URL

QUICK REFERENCE

13

QUICK REFERENCE CARD

WHAT YOU WANT	COMMAND OR SHORTCUT
MOVING AROUND	
`cd ~/claudeprojects`	Go to your projects folder
`cd project-name`	Go into a project
`cd ..`	Go back up one folder
`cd ~`	Go home from anywhere
`pwd`	See where you are
`ls`	See files in current folder

WHAT YOU WANT	COMMAND OR SHORTCUT
`ls -a`	See hidden files too
`mkdir folder-name`	Create a new folder
`mkdir -p folder/subfolder`	Create nested folders
CLAUDE CODE	
`claude`	Start Claude Code
`claude --model opus`	Start with Opus model
`/quit`	Exit Claude Code
`/model`	Switch model during a session
`/compact`	Compact a long session
`/status`	Check current model
KEYBOARD SHORTCUTS	
`Shift + Tab`	Switch modes (normal / auto-edit / plan)
`Ctrl + U`	Erase current line
`Ctrl + C`	Stop a running command
`Up Arrow`	Repeat last command
`clear`	Clear the terminal screen

"command not found: claude"
Close the terminal completely and open a new one. If it still doesn't work, run the install command from Part 1 again.

Claude Code asks you to log in again
Completely normal if it's been a while. Just follow the browser prompts.

Lost and don't know where you are
Type `pwd` to see your location. Type `cd ~` to go home and start fresh.

Claude seems to have forgotten the project
Type `/compact` to compress the session, or start fresh and say "Please read README.md and SNAPSHOT.md before we begin."

Hidden .claude folder isn't showing in Finder
Press **Cmd + Shift + .** while in Finder to reveal hidden files and folders.

Default model keeps reverting to Sonnet
This is a known bug in Claude Code. The most reliable fix is the environment variable method from Part 2 (adding it to your `.zshrc` file).

Written for macOS + Cursor (free). Last updated February 2026.